

AI-POWERED E-COMMERCE PLATFORM WITH INTELLIGENT BUSINESS INSIGHTS

WEEKLY PROJECT UPDATE REPORT 4 - CLASS 7

Abstract - This update integrates Shakib's latest React frontend and Rahat's latest Django backend, restores the missing customer AI service using Django REST Framework, adds a protected admin chatbot connected to live database summaries, and provides a one-click Windows launcher. Both chatbots were tested through direct API requests and the browser interface.

1. SHAKIB - FRONTEND UPDATE

Shakib's latest React/Vite frontend retains API communication for login, registration, products, categories, cart, checkout, order history and the customer chatbot. Product and order pages now request data from Django instead of relying only on local dummy data. Login stores JWT access and refresh tokens. The Login and NotFound pages were redesigned, and the customer chat page continued to call `/api/ai/chat/`.

Integration changes by Farhan: Chat.jsx was improved with loading status, disabled duplicate submissions, visible API errors and multi-line replies. AdminChat.jsx was added, App.jsx received the `/admin-chat` route, and Navbar.jsx received separate Customer AI and Admin AI links.

2. RAHAT - BACKEND UPDATE

Rahat's latest Django backend keeps modular applications for accounts, store, cart, orders, chat and core. Existing CRUD views already use Django REST Framework `ModelViewSet` classes and router-based URLs. The latest archive mainly changes the username limit to 150 characters, adds `STATIC_ROOT`, includes a new migration, and contains sample product/category data. No application file was removed.

The latest backend did not contain the previously working Ollama AI endpoint. Rahat's existing chat app, models and database records were therefore preserved, while AI generation was implemented in a separate chatbot app to avoid overwriting his work.

3. CUSTOMER CHATBOT - DRF INTEGRATION

A new chatbot Django application was created with `serializers.py`, `views.py` and `urls.py`. `ChatRequestSerializer` checks that a non-empty message is supplied. `CustomerChatView` uses DRF `APIView` and `Response` instead of raw Django `JsonResponse` and `csrf_exempt`. The endpoint is `/api/ai/chat/`.

Before calling the local Ollama `phi3:mini` model, Django reads active product names, categories, descriptions, prices and stock from SQLite. The model therefore answers using current catalogue data and is instructed not to invent products, prices or private order information.

4. ADMIN CHATBOT

A second endpoint, `/api/ai/admin-chat/`, was added using the same `phi3:mini` model but a different instruction set. `IsAdminUser` permission and the JWT access token prevent ordinary customers from using this service. The frontend sends the token in the Authorization header.

The admin chatbot receives a read-only database snapshot containing product and category totals, active products, low-stock items, user counts, order counts, order statuses, latest orders, recorded sales values and top-selling items. It can explain the information but cannot create, update or delete database records.

5. STARTUP, TESTING AND GITHUB

`Start-BestCommerce.bat` was added to open separate backend and frontend terminals, activate the Python environment, start Django, start Vite and open the browser automatically. Manual startup commands remain available for demonstration.

Testing covered direct PowerShell POST requests, browser-based customer questions, admin login, protected admin access, low-stock and order queries, customer privacy restrictions, server restart and one-click startup. The updated frontend, backend, launcher and `partsChanged.md` were uploaded only to the FarhanRafid branch; other branches were not modified.

6. FILES ADDED OR MODIFIED

Backend added: `chatbot/serializers.py`, `chatbot/urls.py` and `chatbot/views.py`. Backend modified: `backend/settings.py`, `backend/urls.py` and `requirements.txt`. Frontend added: `src/pages/AdminChat.jsx`. Frontend modified: `src/pages/Chat.jsx`, `src/App.jsx` and `src/components/Navbar.jsx`. Root additions: `Start-BestCommerce.bat` and `partsChanged.md`. No source file from Rahat's or Shakib's latest work was deleted.

7. CURRENT STATUS AND NEXT WORK

The integrated system now demonstrates two working local AI assistants: a public product-aware customer assistant and a protected database-aware admin assistant. Both use the same Ollama model while receiving separate data and permissions. Next work includes stronger API error handling across store pages, token refresh, chat-history storage, human-support takeover and broader end-to-end testing.

REFERENCES AND TOOLS

[1] Django 4.2 Documentation. [2] Django REST Framework `APIView`, `serializers` and `permissions` documentation. [3] Ollama Python library documentation. [4] React, React Router and Vite documentation. [5] Microsoft Windows command documentation. [6] OpenAI ChatGPT used for explanation, debugging and document preparation; all code was locally tested by the team.